



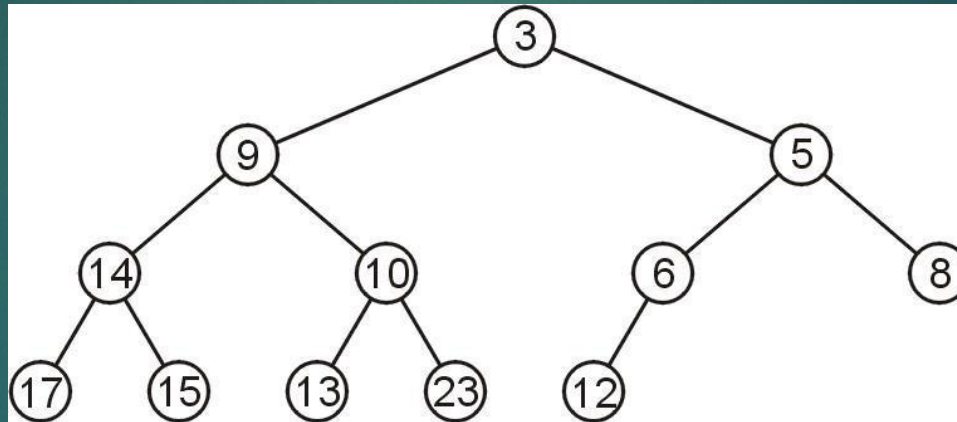
Week 13

Introduction to Heap

Array Storage

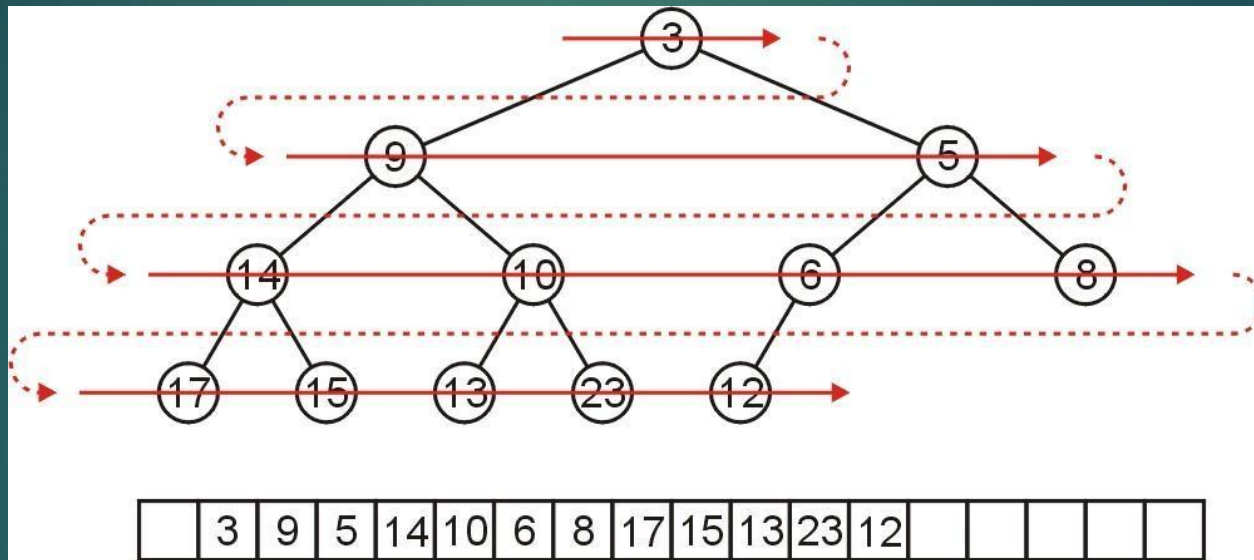
We are able to store a complete tree as an array

- Traverse the tree in breadth-first order, placing the entries into the array



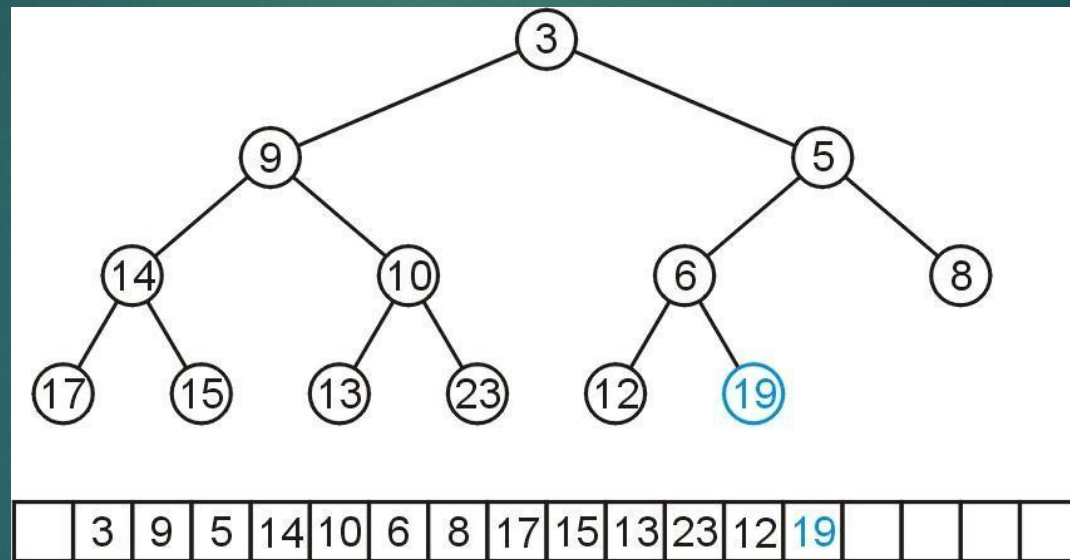
Array Storage (Insertion)

We can store this in an array after a quick traversal:



Array Storage (Insertion)

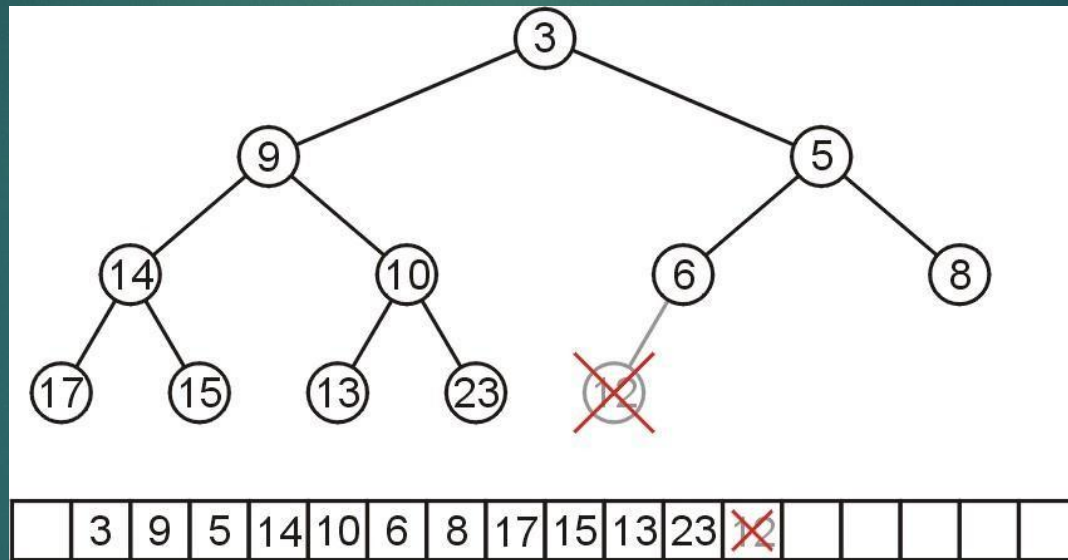
To insert another node while maintaining the complete-binary-tree structure, we must insert into the next array location



Array Storage

(Deletion)

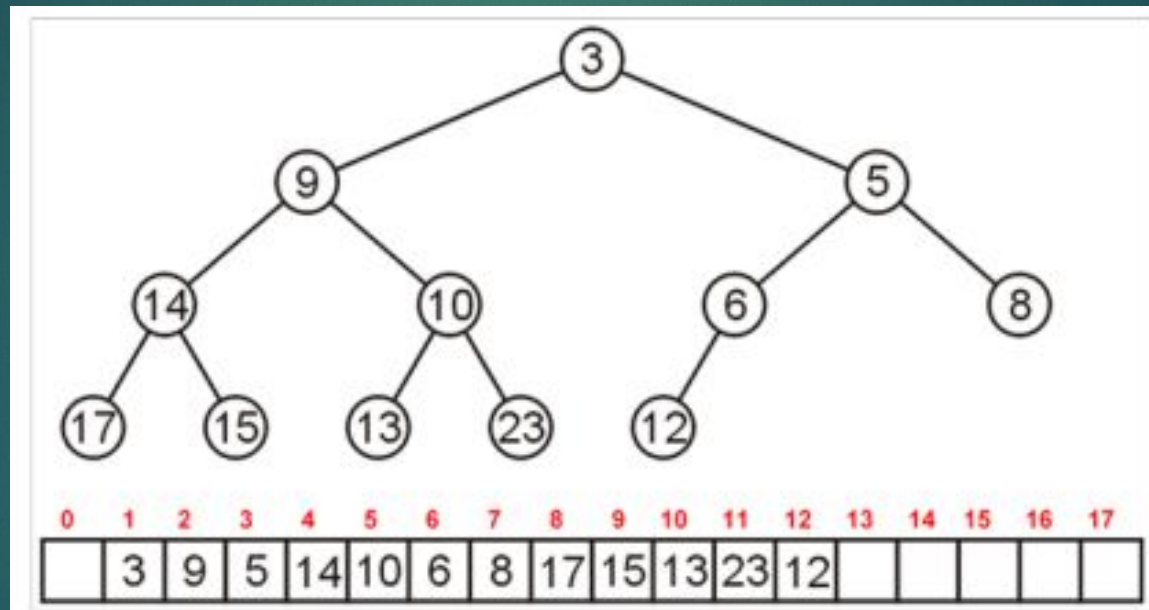
to remove a node while keeping the complete-tree structure, we must remove the last element in the array



Array Storage(Finding Parent and Child Nodes)

Leaving the first entry blank yields a bonus:

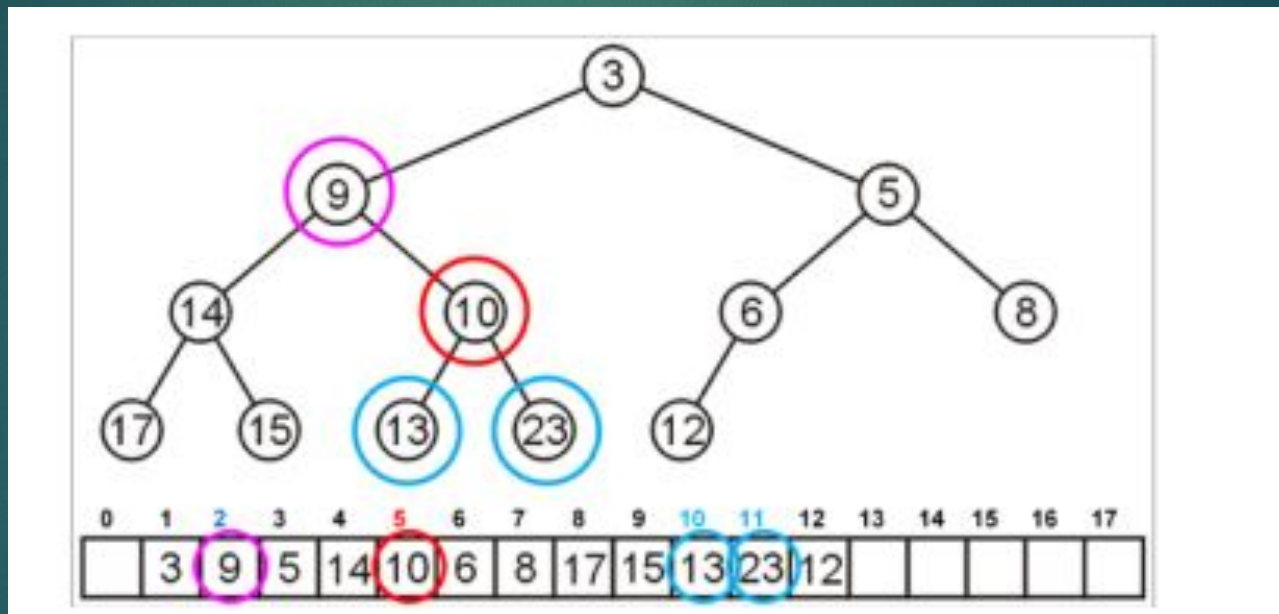
- The children of the node with index k are in $2k$ and $2k + 1$
- The parent of node with index k is in $k \div 2$



Array Storage

For example, node 10 has index **5**:

- Its children 13 and 23 have indices **10** and **11**, respectively



Outline

- Priority Queue
- Examples of Priority Queue
- Implementation details of Priority Queue
- Binary Heap
 - Min Heap
 - Max Heap
- Heap Sort

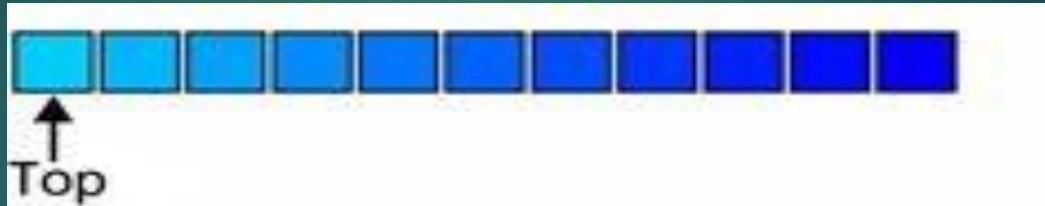
Priority Queue

With Queues

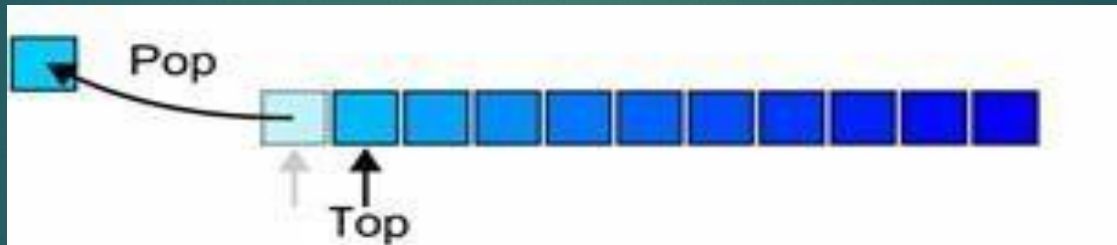
- The order may be summarized by *first in, first out*.
- If each object is associated with a priority, we may wish to pop that object which has highest priority.
- With each pushed object, we will associate a nonnegative integer (0, 1, 2, ...) where:
 - The value 0 has the *highest* priority, and
 - The higher the number, the lower the priority.

Operations

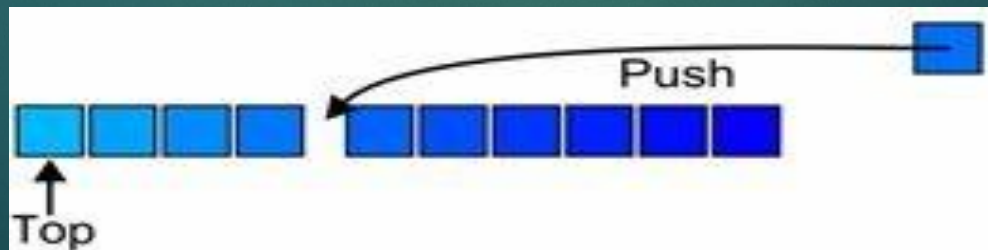
The top of a priority queue is the object with highest priority



Popping from a priority queue removes the current highest priority object:



Push places a new object into the appropriate place



Heap

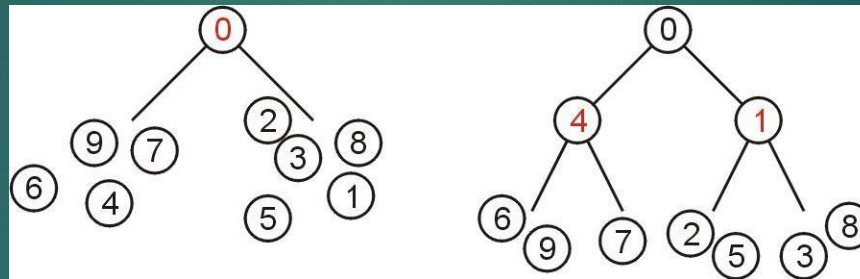
- A **Heap** is a complete binary tree data structure that satisfies the heap property: for every node, the value of its children is greater than or equal to its own value.
- Heaps are usually used to implement priority queues, where the smallest (or largest) element is always at the root of the tree.
- Numerous other heaps exists:
 - D-ary heaps
 - Left-list heaps
 - Skew heaps
 - Binomial heaps
 - Fibonacci heaps
 - Bi-parental heaps

Heap

A non-empty binary tree is a min-heap if

The key associated with the root is less than or equal to the keys associated with either of the sub-trees (if any)

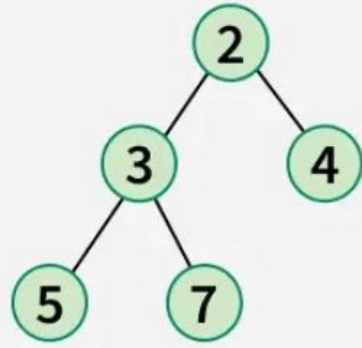
- Both of the sub-trees (if any) are also binary min-heaps



From this definition:

- A single node is a min-heap
- All keys in either sub-tree are greater than the root key

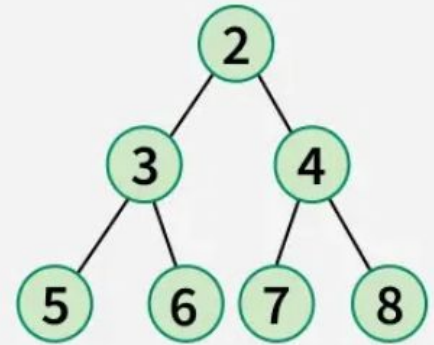
Heap



Min Heap

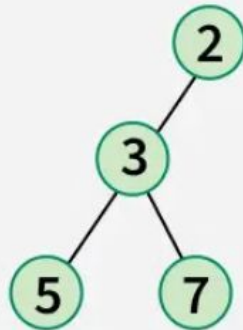


Min Heap

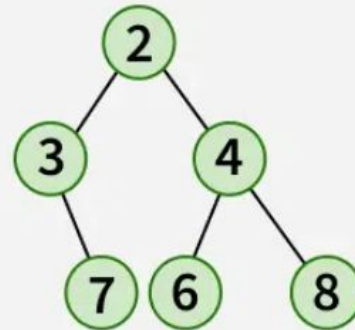


Min Heap

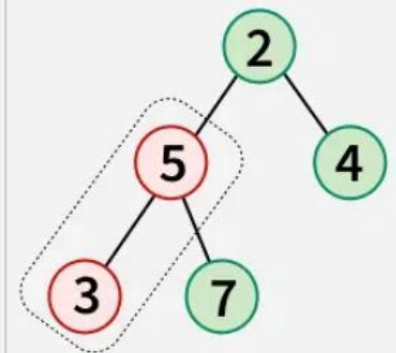
Valid Min Heaps



Not a Complete Binary Tree



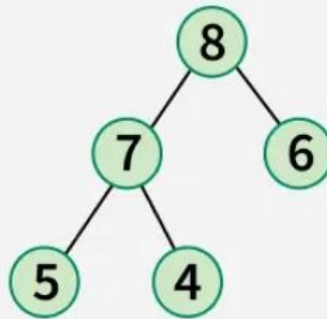
Not a Complete Binary Tree



Violates Min Heap Property

Invalid Min Heaps

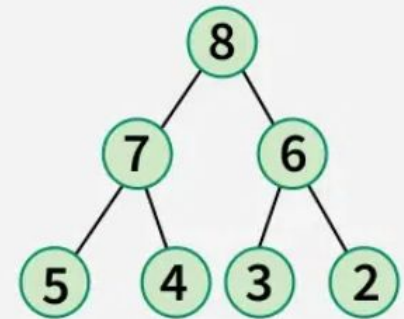
Heap



Max Heap

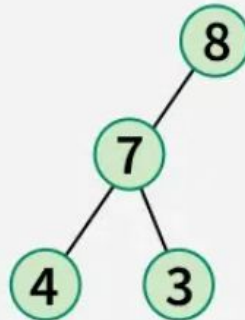


Max Heap

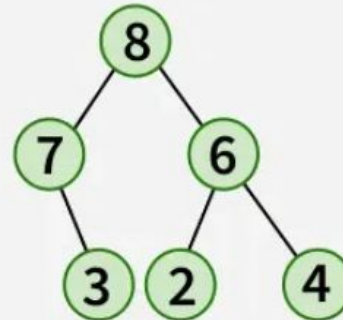


Max Heap

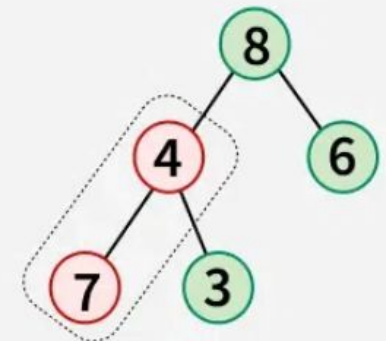
Valid Max Heaps



Not a Complete Binary Tree



Not a Complete Binary Tree

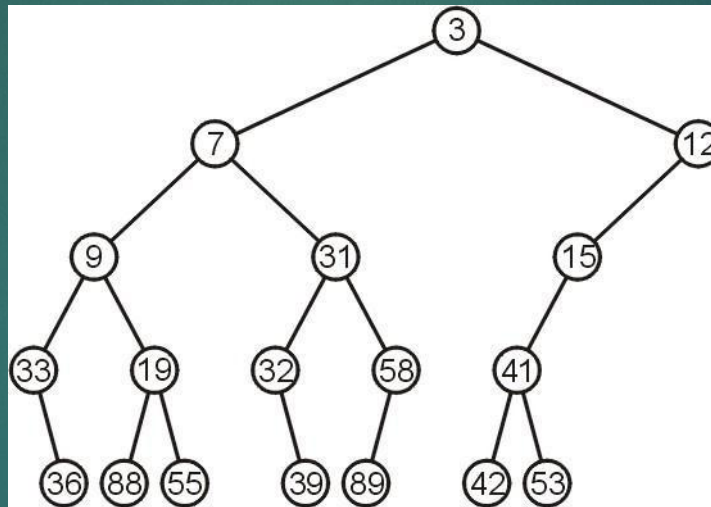


Violates Max Heap Property

Invalid Max Heaps

Example

This is a binary min-heap:



Operations onHeap

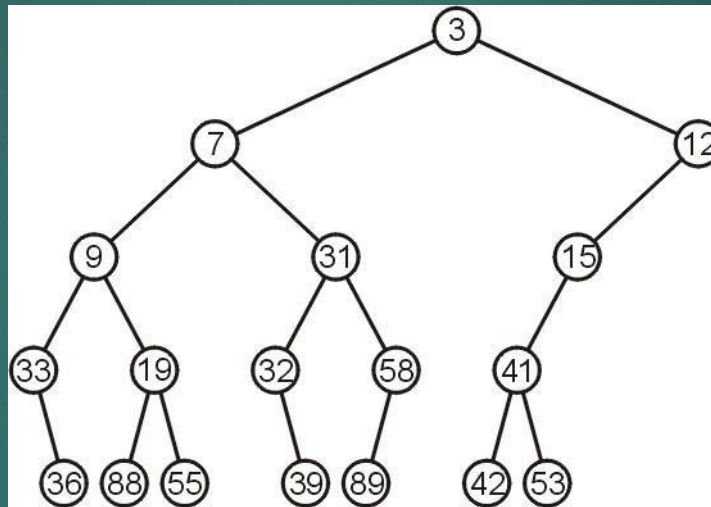


We will consider three operations:

- Top
- Pop
- Push

Example

We can find the top object in $\Theta(1)$ time: 3



Pop

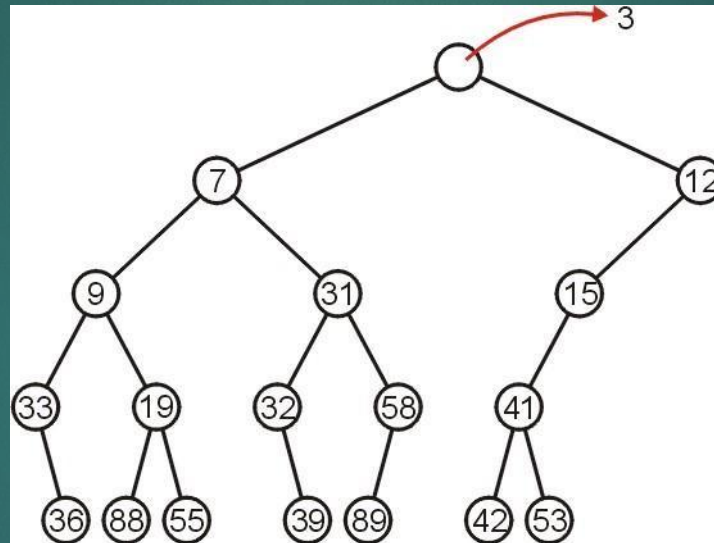
To remove the minimum object:

- Promote the node of the sub-tree which has the least value

Rekurs down the sub-tree from which we promoted the least value

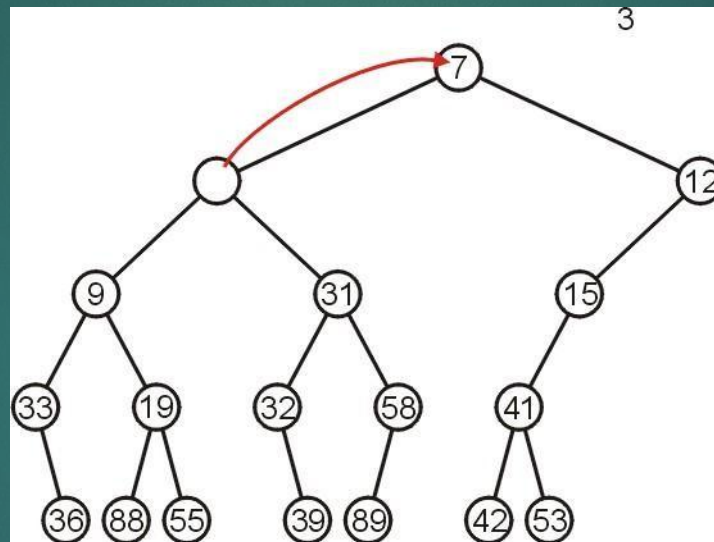
Pop

Using our example, we remove 3:



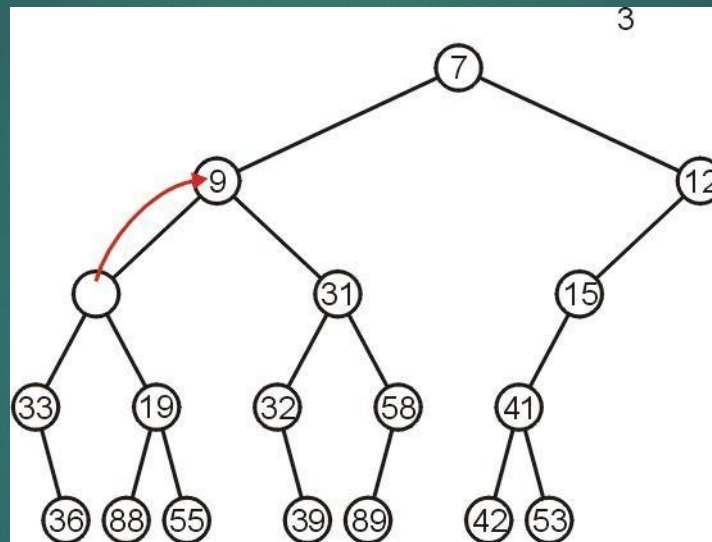
Pop

We promote 7 (the minimum of 7 and 12) to the root:



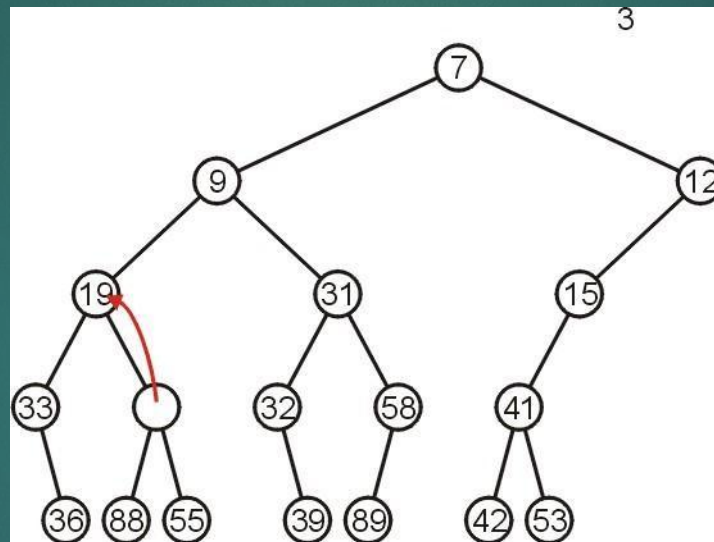
Pop

In the left sub-tree, we promote 9:



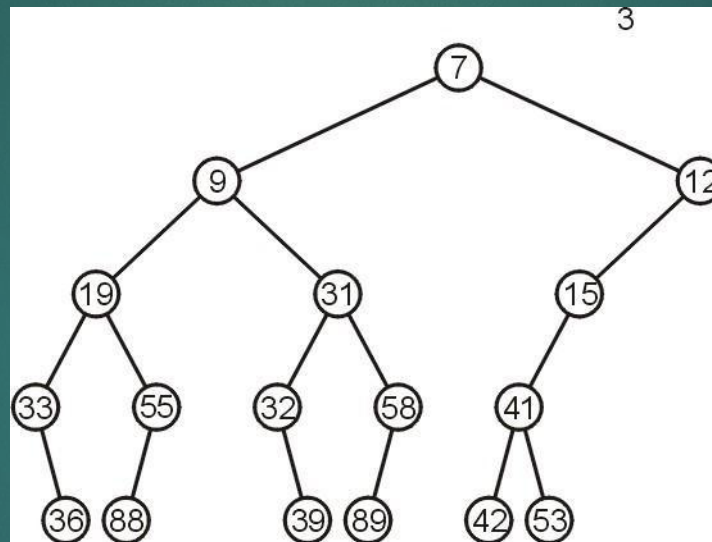
Pop

Recursively, we promote 19:



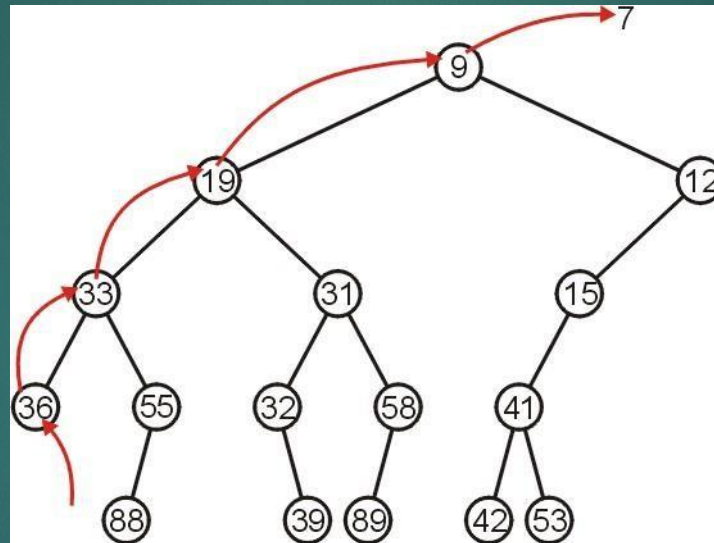
Pop

Finally, 55 is a leaf node, so we promote it and delete the leaf



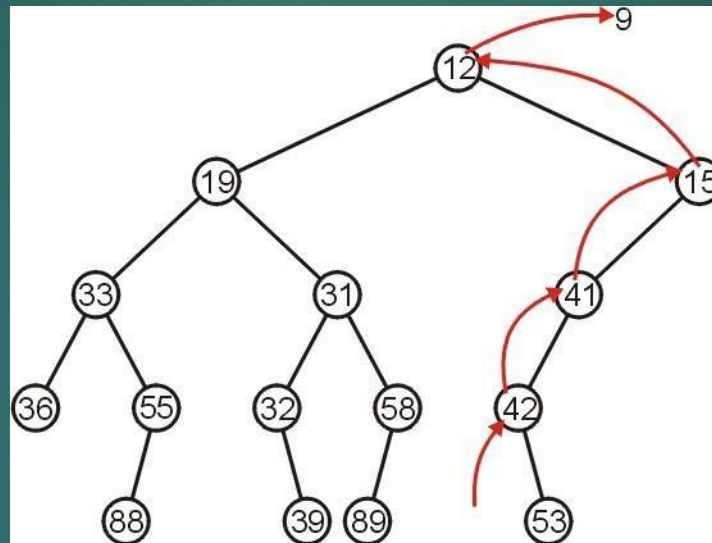
Pop

Repeating this operation again, we can remove 7:



Pop

If we remove 9, we must now promote from the right sub-tree:



Push



Inserting into a heap may be done either:

- At a leaf (move it up if it is smaller than the parent)
- At the root (insert the larger object into one of the subtrees)

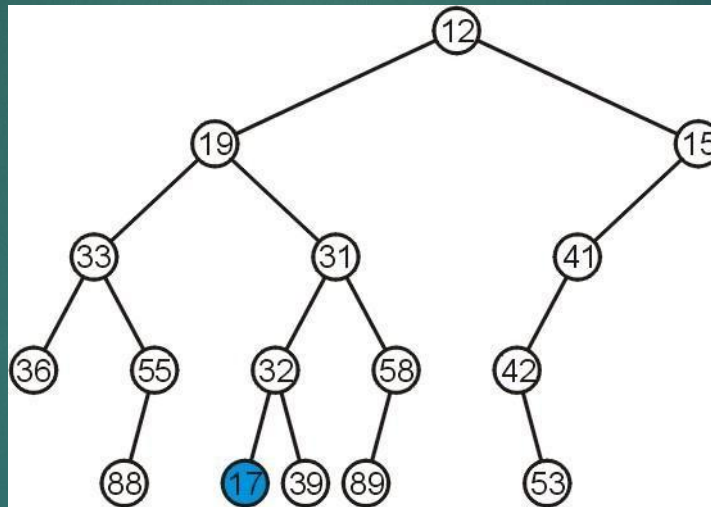
We will use the first approach with binary heaps

- Other heaps use the second

Push

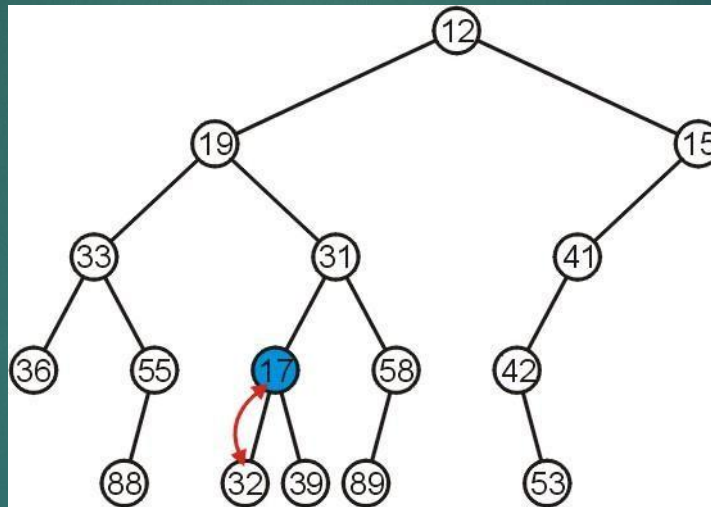
Inserting 17 into the last heap

- Select an arbitrary node to insert a new leaf node:



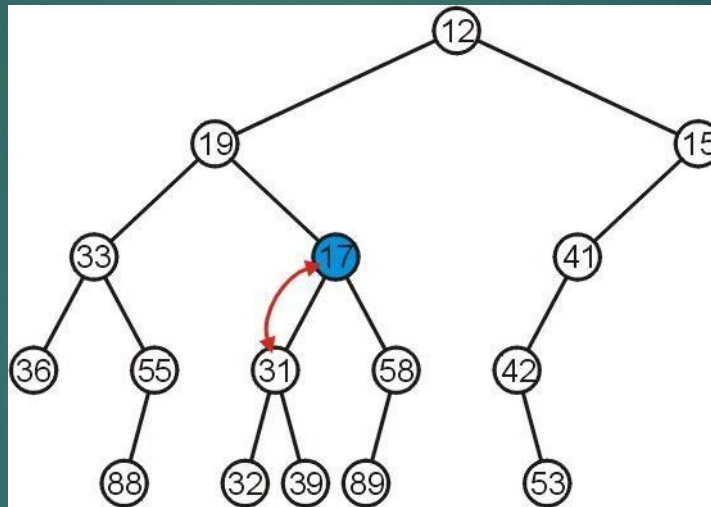
Push

The node 17 is less than the node 32, so we swap them



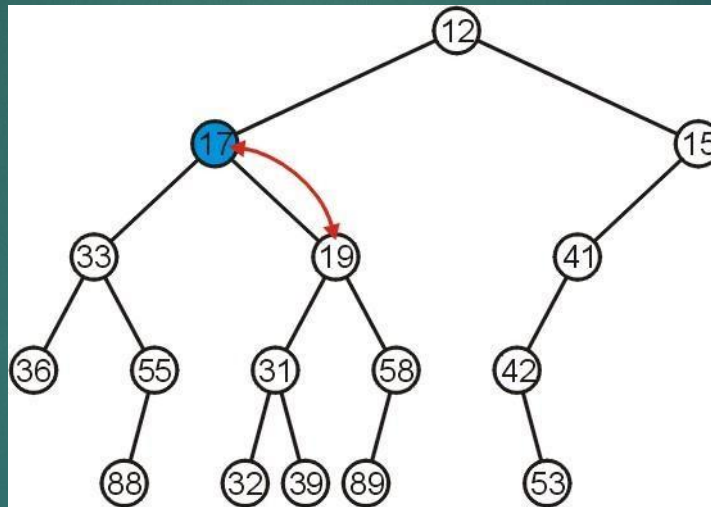
Push

The node 17 is less than the node 31; swap them



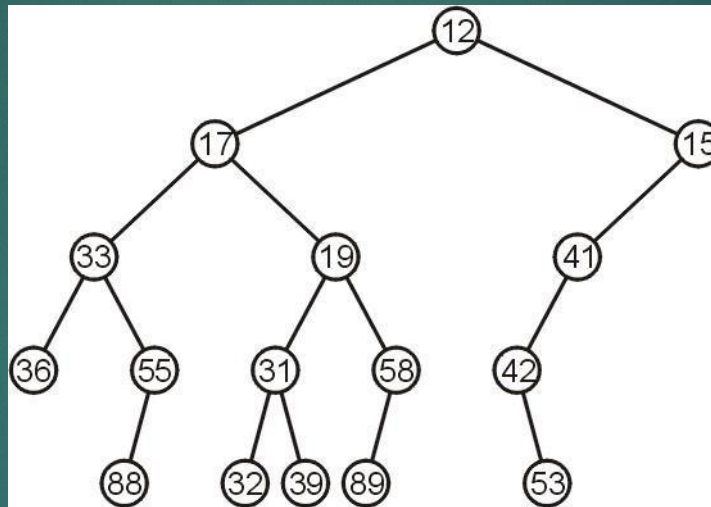
Push

The node 17 is less than the node 19; swap them



Push

The node 17 is greater than 12 so we are finished



Push

Observation: both the left and right subtrees of 19 were greater than 19, thus we are guaranteed that we don't have to send the new node down

This process is called *percolation*, that is, the lighter (smaller) objects move up from the bottom of the min-heap

Implementation Details

By using complete binary trees, we will be able to maintain, with minimal effort, the complete tree structure

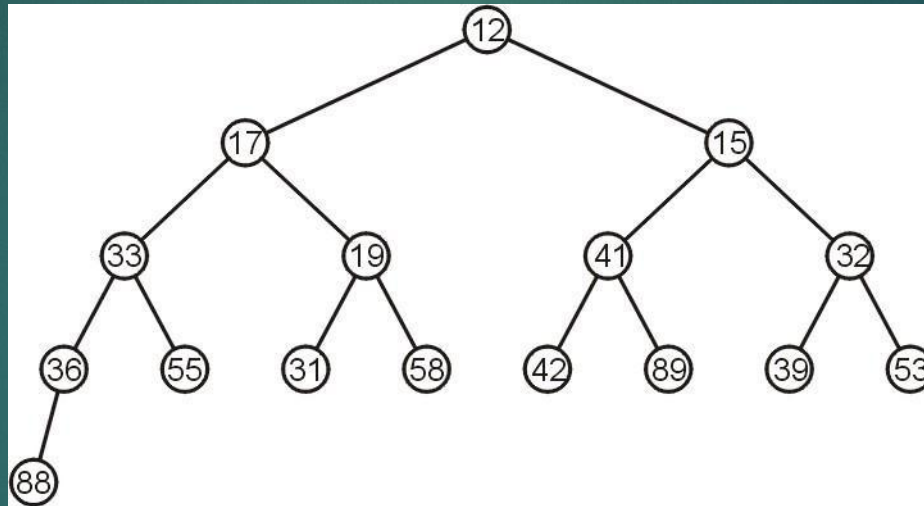
We have already seen

- It is easy to store a complete tree as an array

If we can store a heap of size n as an array of size $\Theta(n)$, this would be great!

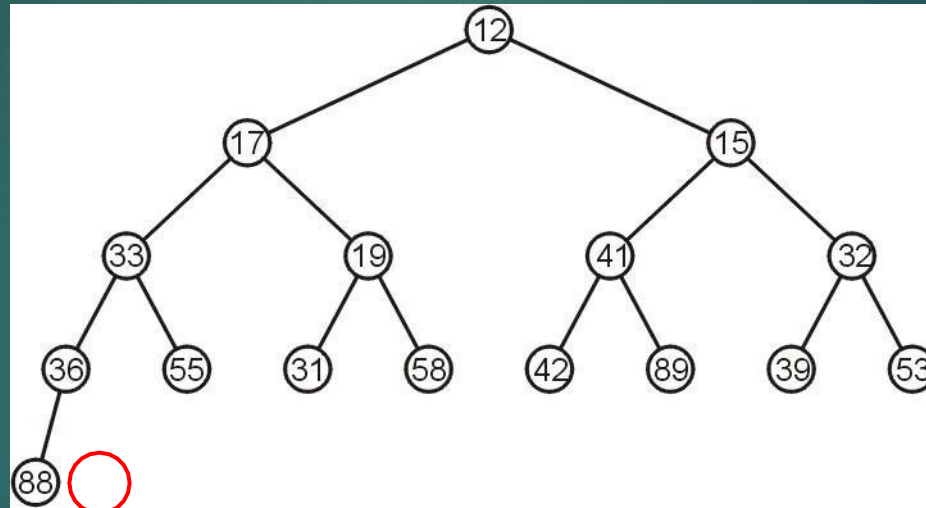
Example

For example, the previous heap may be represented as the following (non-unique!) complete tree:



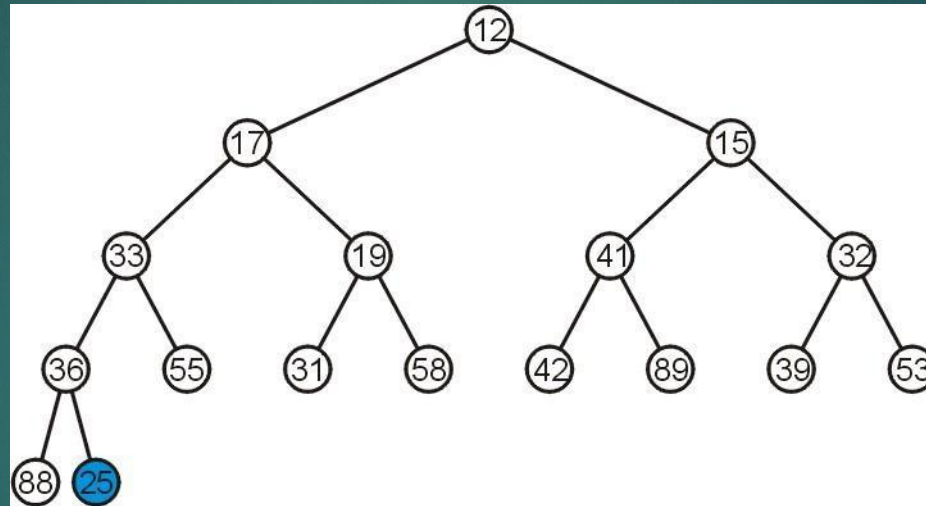
Complete Trees: Push

If we insert into a complete tree, we need only place the new node as a leaf node in the appropriate location and percolate up



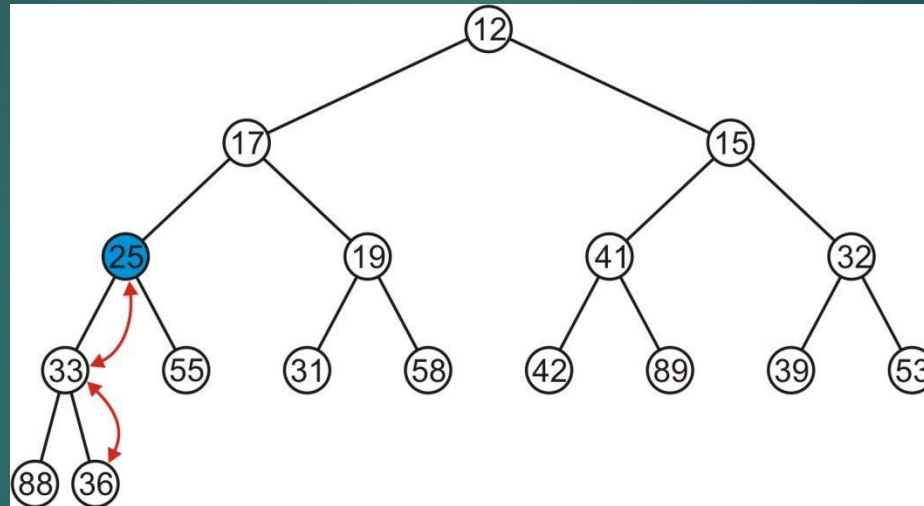
Complete Trees: Push

For example, push 25:



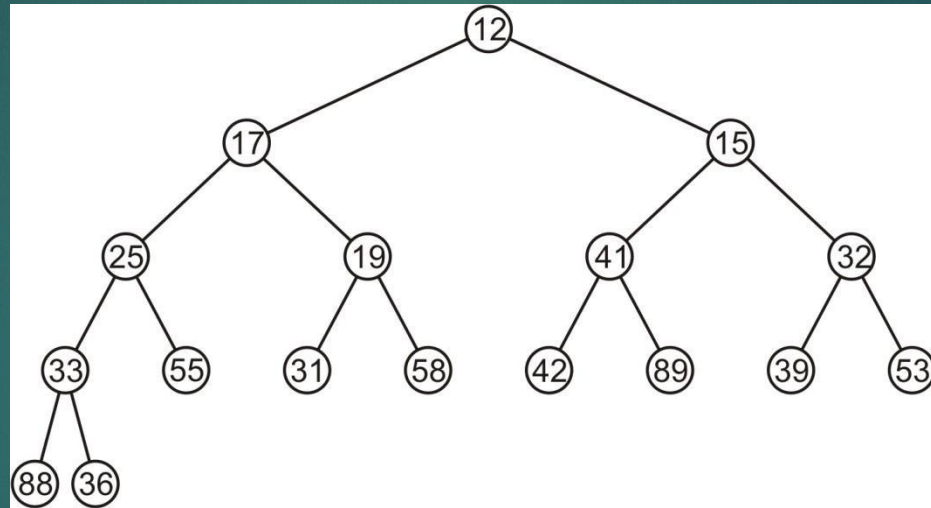
Complete Trees: Push

- We have to percolate 25 up into its appropriate location
- The resulting heap is still a complete tree



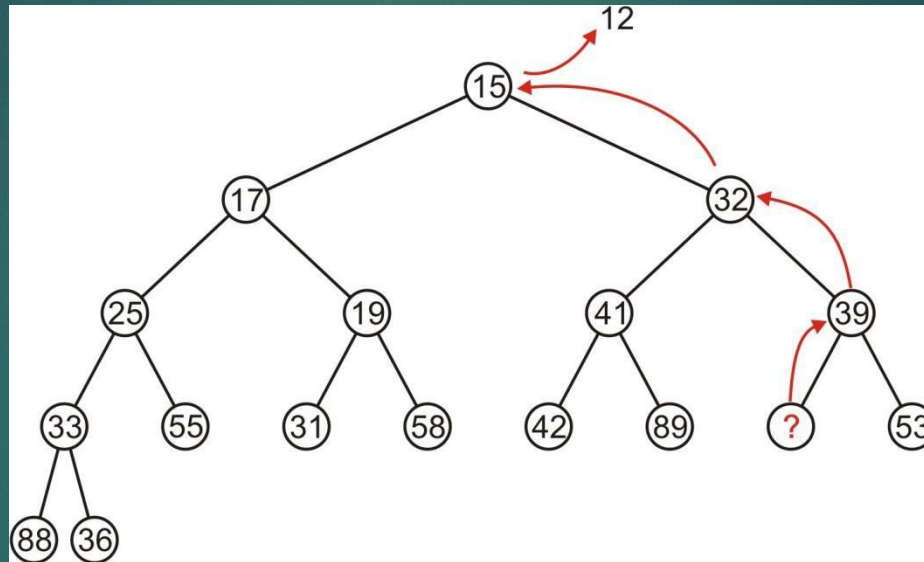
Complete Trees: Push

Suppose we want to pop the top entry:
12



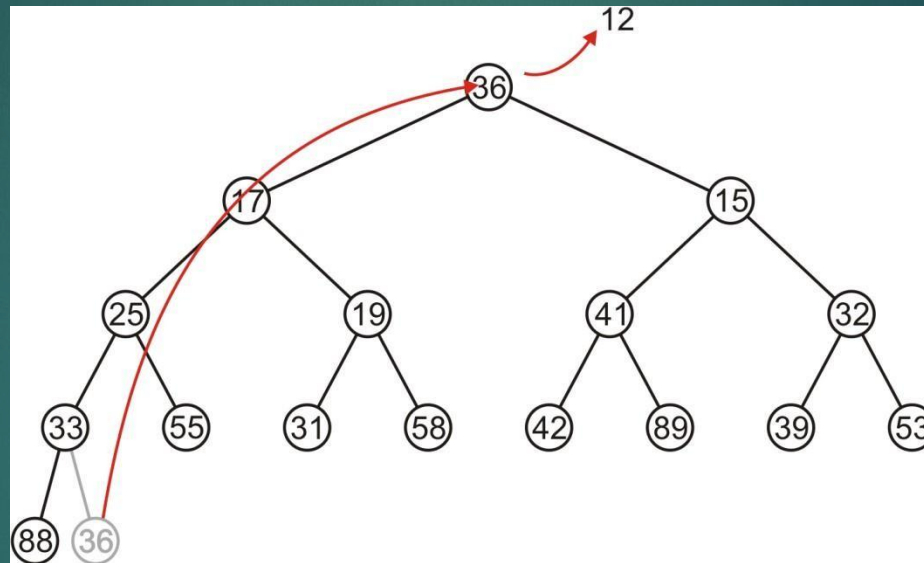
Complete Trees: Pop

Percolating up creates a hole leading to a non-complete tree



Complete Trees: Pop

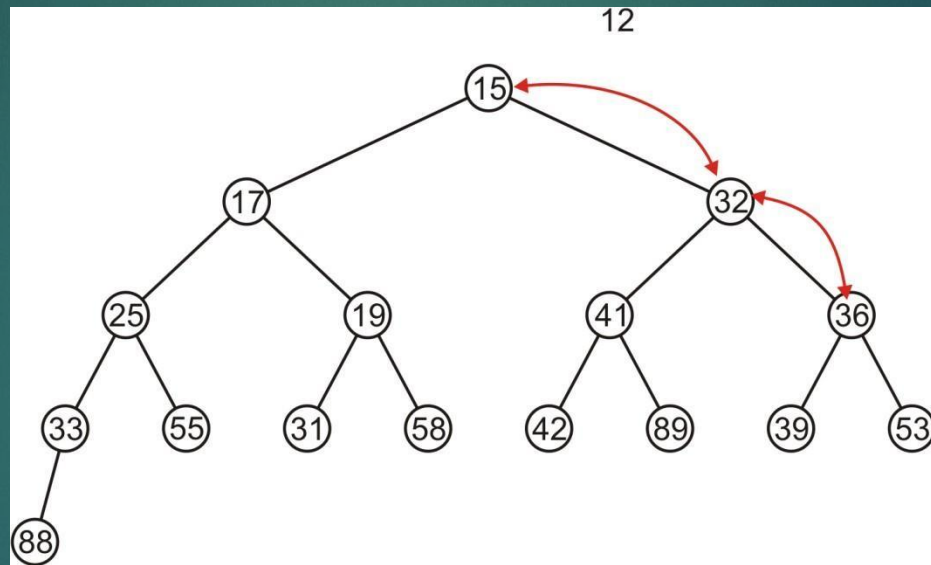
Alternatively, copy the last entry in the heap to the root



Complete Trees: Pop

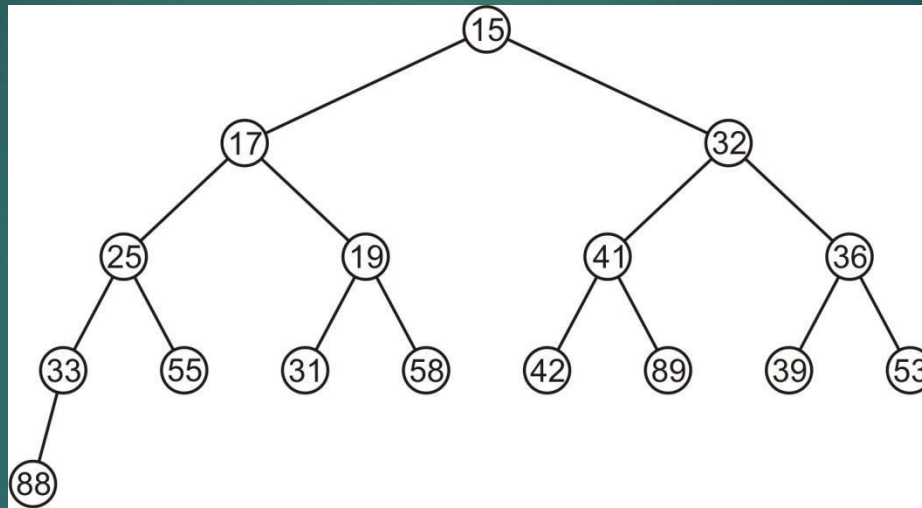
Now, percolate 36 down swapping it with the smallest of its children

- We halt when both children are larger



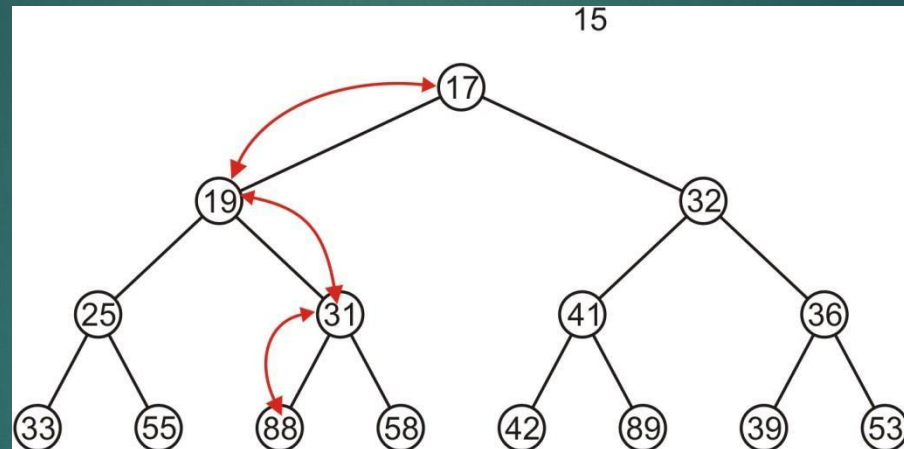
Complete Trees: Pop

The resulting tree is now still a complete tree:



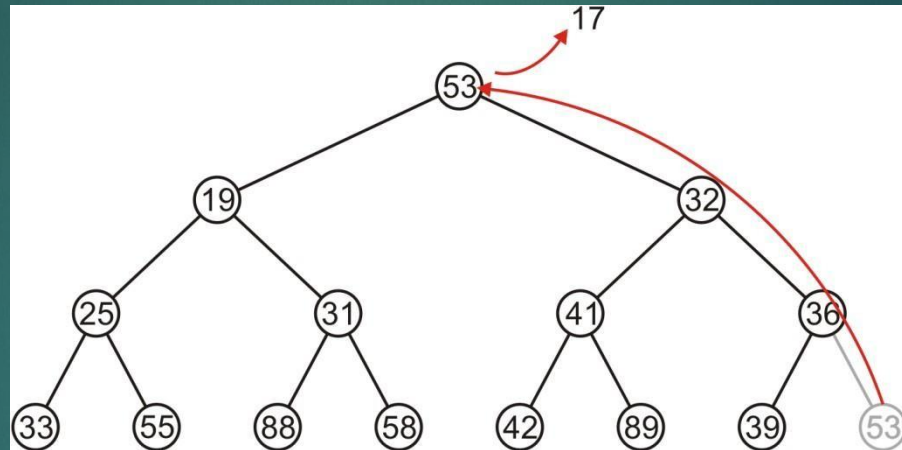
Complete Trees: Pop

This time, it gets percolated down to the point where it has no children



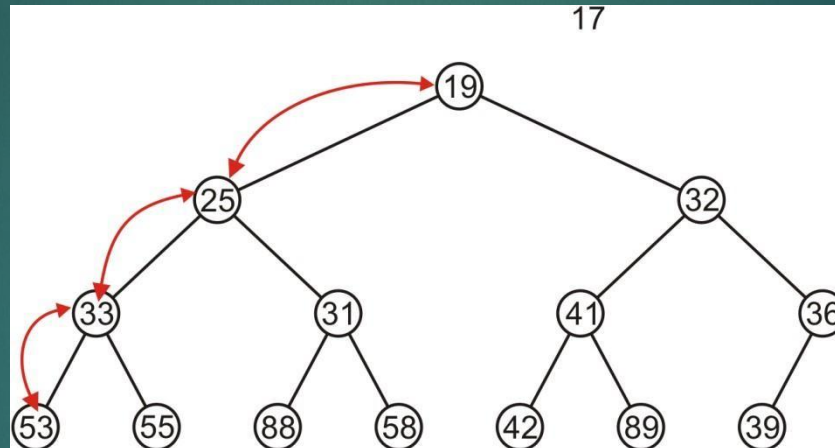
Complete Trees: Pop

In popping 17, 53 is moved to the top



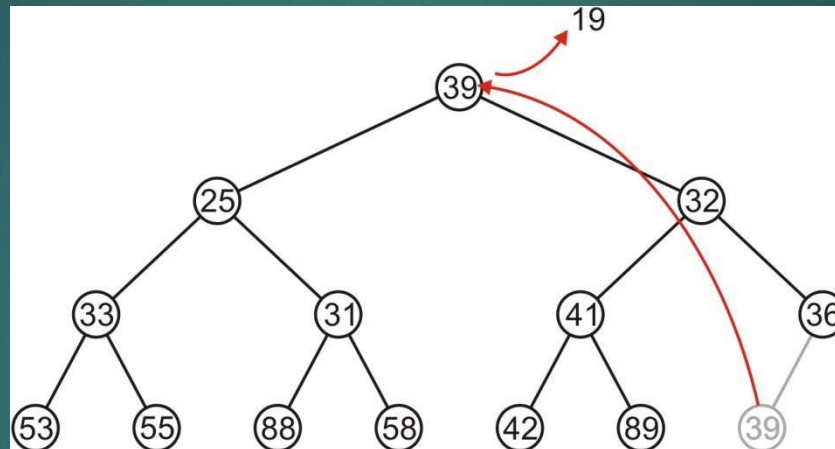
Complete Trees: Pop

And percolated down, again to the deepest level



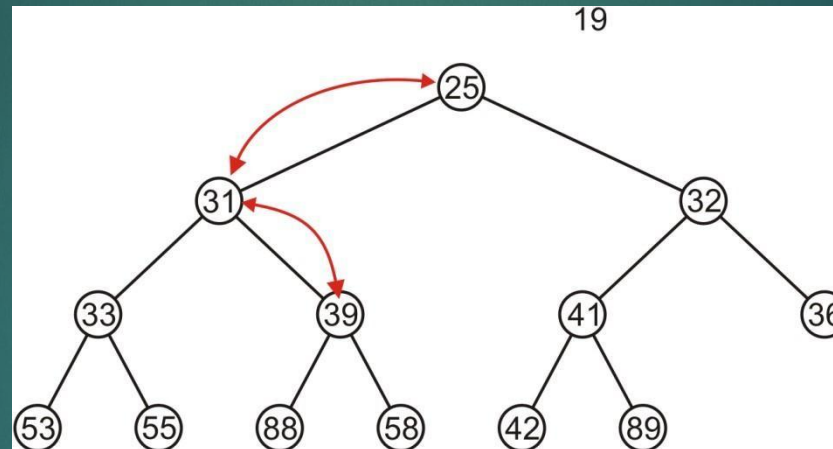
Complete Trees: Pop

Popping 19 copies up 39



Complete Trees: Pop

Which is then percolated down to the second deepest level



Complete Tree

- Therefore, we can maintain the complete-tree shape of a heap
- We may store a complete tree using an array:
 - A complete tree is filled in breadth-first traversal order
 - The array is filled using breadth-first traversal
- This structural requirement, along with the heap property, is essential for the heap data structure to guarantee **efficient $O(\log n)$ time complexity** for insertion and deletion operations, and for efficient memory representation using an array.
- Without the complete binary tree property, the tree could become unbalanced, degrading performance in the worst case to $O(n)$ time, similar to a linked list.

Heap Sort

What is Heap Sort?

Heap Sort is a **comparison-based sorting algorithm** that uses a **binary heap** data structure.

It first builds a **max heap** and then repeatedly extracts the **maximum element** (root) to sort the array in **ascending order**.

Key Idea:

A **max heap** is a complete binary tree where each node is **greater than or equal to its children**.

Heap Sort leverages this property to sort elements efficiently.

Heap Sort

Algorithm Steps:

1. Build Max Heap

1. Convert the array into a max heap.
2. The largest element will be at the root (index 0).

2. Extract Max Element

1. Swap the root (largest element) with the **last element**.
2. Reduce the heap size by 1 (ignore the last element, as it's now sorted).

3. Heapify

1. Restore the max heap property for the remaining heap.
2. Repeat steps 2–3 until all elements are sorted.

Heap Sort

- Discussion

0	1	2	3	4	5	6	7	8	9
	3	1	4	7	8	6	0	2	5

- Step 01: Build Min Heap/Max Heap
- Step 02: Swap First index with Last index
- Step 03: Update heap size(HS)
- Step 04: Apply heapify on HS

Heap Sort

Example Dry Run

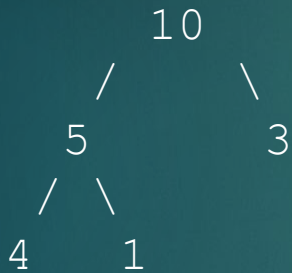
Let's sort the array:

[4, 10, 3, 5, 1]

Step 1: Build Max Heap

After heapify → [10, 5, 3, 4, 1]

Heap (level-wise):



Step 2: Extract max (10)

Swap root with last → [1, 5, 3, 4, 10]

Heapify [1, 5, 3, 4] → [5, 4, 3, 1, 10]

Step 3: Extract next max (5)

Swap → [1, 4, 3, 5, 10]

Heapify [1, 4, 3] → [4, 1, 3, 5, 10]

Heap Sort

Step 4: Extract next max (4)

Swap $\rightarrow [3, 1, 4, 5, 10]$

Heapify $[3, 1] \rightarrow [3, 1, 4, 5, 10]$

Step 5: Extract next max (3)

Swap $\rightarrow [1, 3, 4, 5, 10]$

Final Sorted Array: $[1, 3, 4, 5, 10]$

Operation	Complexity
Building heap	$O(n)$
Each heapify	$O(\log n)$
Overall sorting	$O(n \log n)$
Space complexity	$O(1)$ (in-place)

Heap Sort

Advantages

- Efficient for large datasets.
- In-place sorting (no extra memory).

Disadvantages

- Not stable (equal elements may change order).
- Slightly slower in practice than Quick Sort due to more swaps.

Run-time Analysis

Accessing the top object is $\Theta(1)$

Popping the top object is $O(\ln(n))$

We copy something that is already in the lowest depth—it
will likely be moved back to the lowest depth

Priority Queues

Now, does using a heap ensure that that object in the heap which:

- has the highest priority, and
- of that highest priority, has been in the heap the longest

Consider inserting seven objects, all of the same priority (colour indicates order):

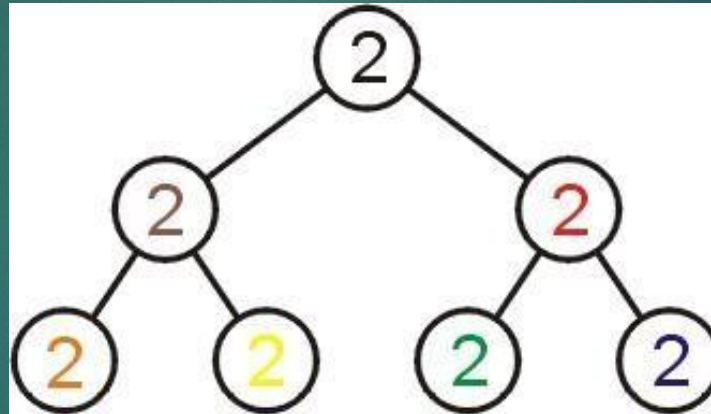
2, 2, 2, 2, 2, 2, 2

Priority Queues

Whatever algorithm we use for promoting must ensure that the first object remains in the root position

= Thus, we must use an insertion technique where we only percolate up if the priority is lower

The result:



Challenge:

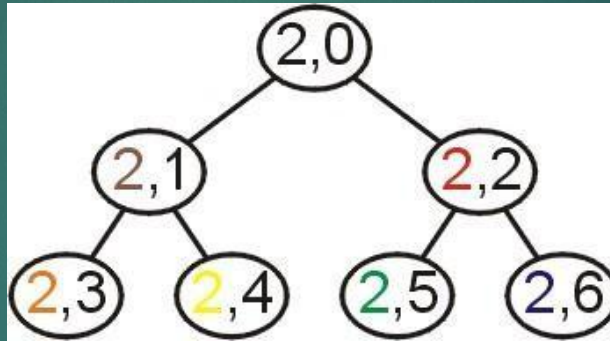
- Come up with an algorithm which removes all seven objects in the original order

Lexicographical Ordering

A better solution is to modify the priority:

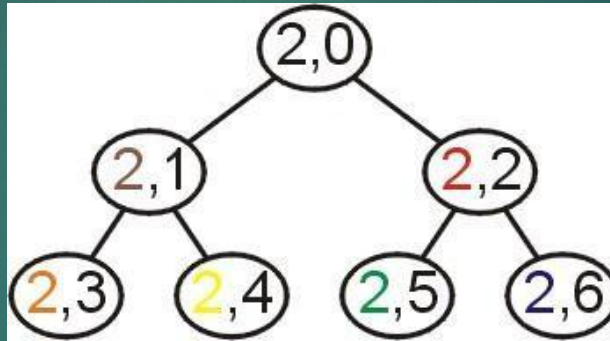
- Track the number of insertions with a counter k (initially 0)
- For each insertion with priority n , create a hybrid priority (n, k) where:

$(n_1, k_1) < (n_2, k_2)$ if $n_1 < n_2$ or $(n_1 = n_2 \text{ and } k_1 < k_2)$



Priority Queues

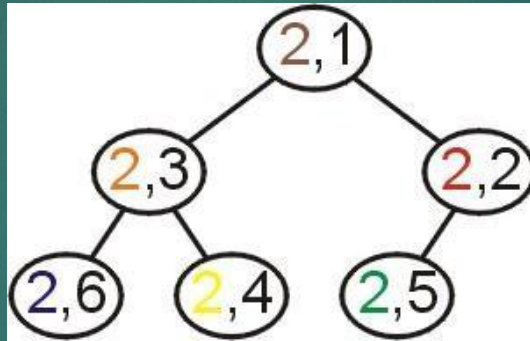
Removing the objects would be in the following order:



Priority Queues

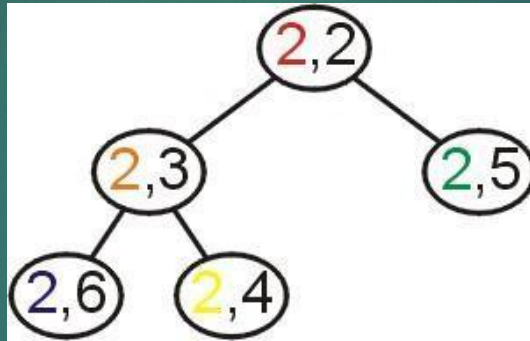
Popped: 2

- First, $(2,1) < (2,2)$ and $(2,3) < (2,4)$



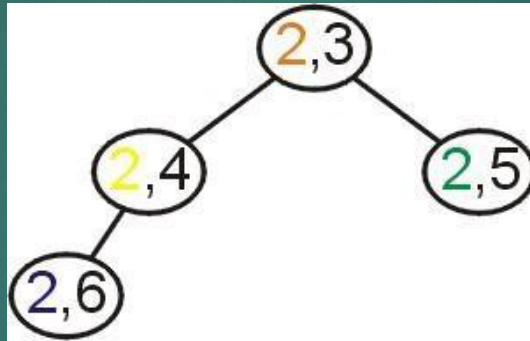
Priority Queues

Removing the objects would be in the following order:



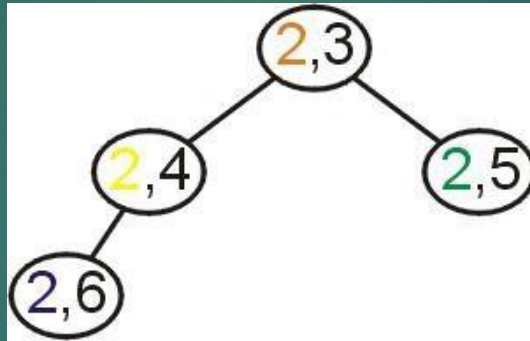
Priority Queues

Removing the objects would be in the following order:



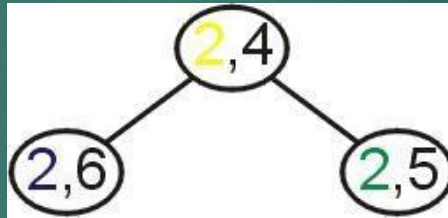
Priority Queues

Removing the objects would be in the following order:

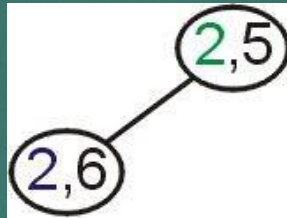


Priority Queues

Removing the objects would be in the following order:



Priority Queues



Summary



In this talk, we have:

- Discussed binary heaps
- Looked at an implementation using arrays
- Discussed implementing priority queues using binary heaps
- Discussed Heap Sort
- The use of a lexicographical ordering